

Laudo de Robustez: Preservação Total no Chunker, Escopo de Sessão e Self-Check Determinístico

Projeto Assistente de Editais — Branch `deve1op` — Abril 2026

1. Contexto

O laudo 07 documentou a virada arquitetural para um pipeline de Markdown estruturado gerado por LLMs multimodais, com chunker estrutural sobre headers `##` e ferramentas de agente reorganizadas em torno da nova representação. A bateria de avaliação reportada ali, restrita ao edital BNDES, atingiu 8,5/10 e identificou que as duas falhas remanescentes (vagas PcD e comparação de datas) não eram estruturais e dependiam apenas de o agente seguir o procedimento operacional padrão.

Este laudo documenta a primeira aplicação dessa arquitetura aos **três editais simultaneamente** — BNDES, CVM e Petrobras — e os ajustes executados em consequência das falhas observadas. Três classes de problema vieram à tona, cada uma exigindo um tratamento diferente:

- um **bug estrutural no chunker** que descartava texto silenciosamente, fazendo regras inteiras do edital sumirem do índice;
- um **vazamento de identidade de edital** através do histórico de tool calls, que levava o agente a consultar o edital errado em conversas com múltiplos turnos;
- uma **fragilidade nas ferramentas** que dependiam de pré-extração estruturada por LLM, que retornavam dados parciais e induziam o agente a respostas de ausência incorretas.

Como mecanismo de defesa adicional contra alucinação, foi incorporada uma camada determinística de verificação acionada quando o agente responde sem consultar nenhuma ferramenta no turno corrente.

2. Sintomas observados

A avaliação inicial sobre os três editais expôs respostas erradas em perguntas cuja resposta correta estava demonstrável no PDF original. Três casos foram representativos dos tipos de falha encontrados.

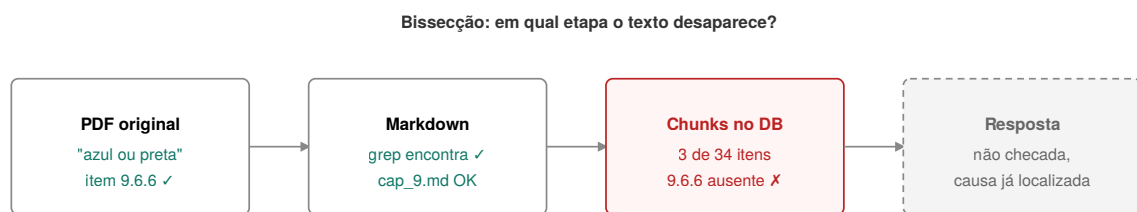


Figura 1. Os três sintomas representativos. Cada caso corresponde a uma causa raiz distinta — bug de chunker, fragilidade de ferramenta pré-computada, e alucinação contra contexto recuperado.

A diferença entre as causas só ficou clara após investigação direta no índice ChromaDB. Investigar o trajeto de cada informação — do PDF original ao Markdown gerado, do Markdown aos chunks indexados, dos chunks à resposta — foi o que permitiu separar problemas de natureza diferente que, à primeira vista, pareciam o mesmo "agente respondeu errado".

3. Princípio de preservação total no chunker

Os casos 1 (caneta) e 2 (taxa) compartilharam a mesma causa raiz, ainda que com manifestações diferentes. A investigação seguiu uma cadeia de quatro perguntas de bissecção, ilustrada na figura abaixo: o texto está no PDF? está no Markdown gerado pelo LLM? está em algum chunk no ChromaDB? está em chunk recuperável pela busca semântica?



A perda ocorre entre o Markdown e o índice — o chunker é o ponto de falha.

Figura 2. Estratégia de diagnóstico. A bissecção entre etapas permite atribuir a falha ao componente correto sem suposições.

3.1 O bug

No caso da caneta, `grep` no Markdown do capítulo 9 da CVM mostrou três itens distintos contendo a regra (9.6.6, 9.7.4, 10.1). Já o `get` no ChromaDB filtrado por capítulo 9 retornou apenas três chunks de um total de 34 itens estruturais no Markdown — uma perda de mais de 30 itens.

A análise passo a passo do `md_chunker.py` localizou o ponto exato do descarte. A função `_parse_blocos` produzia 44 blocos a partir do Markdown do capítulo. O filtro:

```
blocos = [b for b in blocos if b["conteudo"].strip()]
```

reduzia esses 44 a 6. O motivo: o LLM Anthropic, ao transcrever os itens curtos do edital, geralmente coloca a regra inteira dentro do próprio header, sem quebra de linha. O resultado é um bloco com `header` substantivo e `conteudo` vazio.

Como o LLM transcreve um item curto do edital

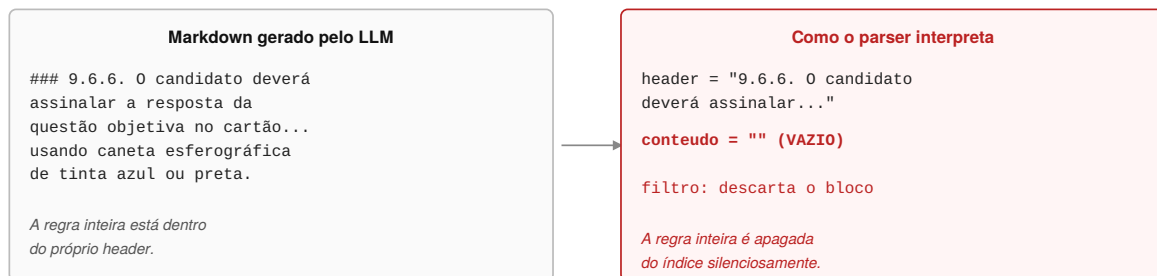


Figura 3. O bug do filtro. O parser supõe que header e conteúdo estão separados, mas o LLM funde os dois em itens curtos. O filtro descarta o bloco como "vazio" mesmo carregando texto substantivo no header.

O LLM, repare-se, transcreveu o documento com fidelidade: itens curtos de edital são, semanticamente, **parágrafos numerados**, não títulos de seção. O bug estava no chunker, que assumia uma separação rígida entre rótulo de seção e conteúdo que o documento real não respeita.

3.2 Primeira correção: mover texto preso no header

A primeira tentativa introduziu uma função intermediária `_mover_header_longo_para_conteudo`, que detecta blocos com `conteudo` vazio cujo `header` contém numeração hierárquica seguida de texto com mais de vinte caracteres, e move esse texto do campo `header` para o campo `conteudo` antes do filtro.

O efeito sobre o índice foi imediato:

Edital	Antes	Depois (correção 1)	Ganho
CVM	32 chunks	119 chunks	+272%
BNDES	52 chunks	138 chunks	+165%
Petrobras	91 chunks	141 chunks	+55%

A correção resolveu o caso da caneta. Mas o caso da taxa (item 5.1 da Petrobras) resistiu.

3.3 O caso que escapou: limiar arbitrário

O item 5.1 do edital tem o formato `## 5.1 TAXA: R$ 79,83.` — apenas **15 caracteres** após a numeração, abaixo do limiar de 20 da função intermediária. O filtro continuava descartando.

Observação: qualquer limiar é arbitrário. Baixar para 5 captura este caso, mas começa a transformar subtítulos genuínos em conteúdo (ex.: "DA PROVA OBJETIVA" tem 17 caracteres). Subir o limiar perde casos curtos. Não há valor correto.

3.4 Segunda correção: princípio invariante

A discussão deixou claro que a correção precisava sair do regime de heurísticas e voltar a um princípio invariante: **nenhum texto presente no Markdown pode ser descartado pelo chunker, em qualquer circunstância**. Operacionalmente, o filtro foi reescrito para preservar o bloco se *qualquer* dos campos contiver texto:

```
blocos = [b for b in blocos if b["conteudo"].strip() or b["header"].strip()]
```

A renderização final do chunk via `_bloco_para_md` já injetava a cadeia ancestral e o conteúdo de forma que o texto presente em qualquer campo aparecesse no Markdown emitido. O conserto foi cirúrgico: uma única condição lógica passou de um *AND* implícito para um *OR* explícito.

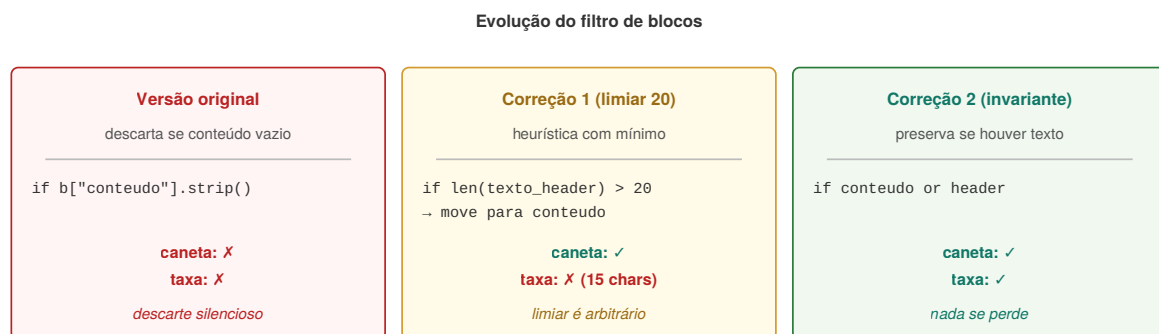


Figura 4. As três versões do filtro. Da heurística arbitrária à invariante explícita: a regra deixa de ser "descarte se vazio" e passa a ser "preserva se há texto em qualquer campo".

A função intermediária `_mover_header_longo_para_conteudo` foi mantida porque ainda agrega valor ao caso de blocos longos: ela coloca a numeração isolada no `header` e o texto no `conteudo`, o que é proveitoso para o embedding e para a renderização. Mas ela *deixou de ser o mecanismo de salvamento* — o filtro permissivo é a garantia.

4. Escopo de edital fixado pela interface

O segundo bug não era sobre conteúdo, mas sobre *roteamento*. Em uma conversa contendo várias perguntas em sequência, o agente ocasionalmente respondia consultando o edital errado: o usuário perguntava sobre a CVM e o agente buscava em `edital_id=1` (BNDES). A trace no LangSmith confirmou: a tool `buscar_no_edital` era invocada com o ID incorreto.

4.1 A causa: vazamento via histórico

A causa estava na arquitetura de mensagens do agente. O `chat_history` passado a cada turno contém todas as mensagens anteriores, inclusive as `AIMessage` com `tool_calls`. Quando o

agente, em um turno anterior, tinha chamado a tool com `edital_id=1`, esse parâmetro permanecia visível no histórico. No turno seguinte, o LLM tendia a reutilizar o mesmo `edital_id` por economia, sem reconsiderar a mudança de contexto da nova pergunta. É um padrão de cache mental que LLMs com histórico longo exibem com frequência.

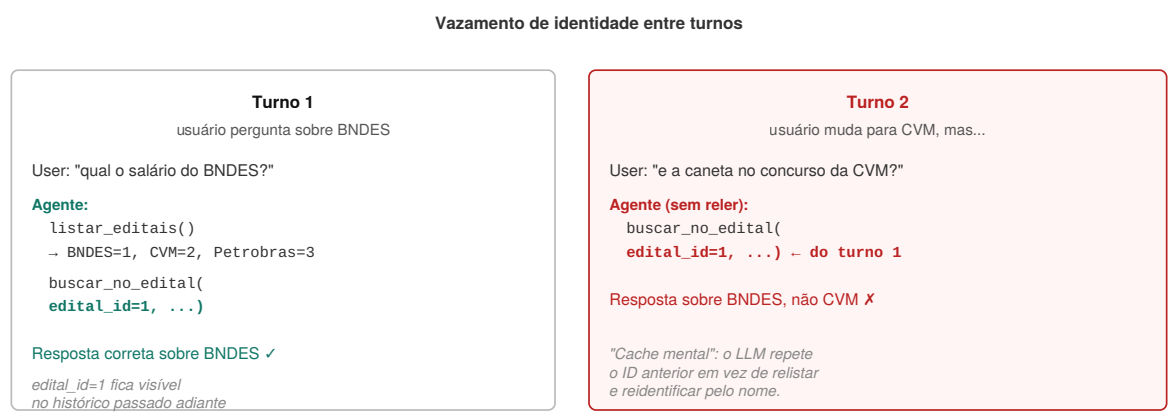


Figura 5. O vazamento. O `edital_id` do turno anterior fica disponível no histórico e é reutilizado pelo LLM mesmo quando o assunto muda.

4.2 Direções consideradas

Direção	Vantagem	Trade-off
(a) Reforçar o prompt com regra anti-cache	Mínima alteração de código	LLM ignora a regra com frequência
(b) Filtrar tool_calls do histórico passado	Maior robustez no agente	Cada turno repete <code>listar_editais</code> ; latência cresce
(c) Fixar o edital ativo no estado da sessão	Robusto e explícito	Perde comparação multi-edital em uma só conversa

4.3 Solução adotada: escopo na UI

A direção (c) foi adotada. A justificativa é conceitual: a identidade do edital deixou de ser uma variável a ser inferida pelo agente a cada turno e passou a ser uma **propriedade da sessão**. A escolha do edital é responsabilidade da interface, não do agente. Esse é o padrão estabelecido em chats de documento (Notion AI, ChatPDF, NotebookLM): o documento ativo é fixado antes da conversa começar.

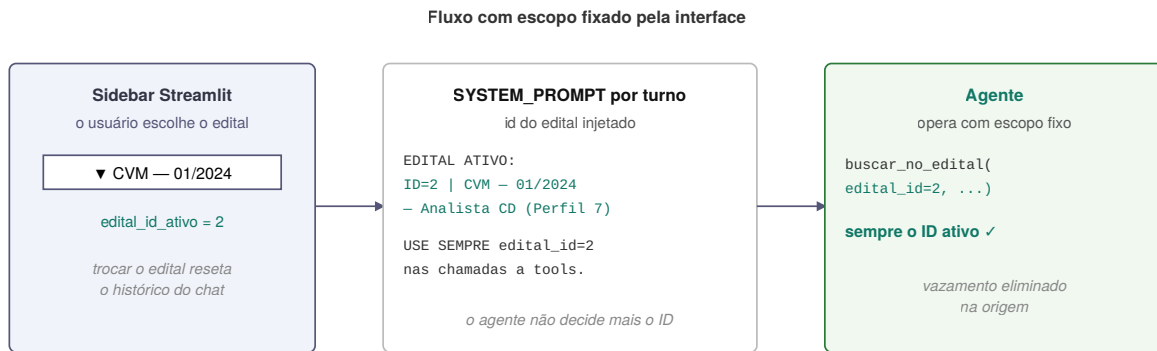


Figura 6. O escopo do edital se torna estado da sessão. O agente perde a responsabilidade de escolher o ID; a UI cuida disso, e a troca reseta a conversa.

A perda de capacidade de comparação multi-edital numa única conversa é assumida como trade-off. Antes da correção, essa capacidade já era aparente: o agente eventualmente respondia certo, mas com alta variância. Tornar o escopo explícito eliminou a variância em troca de uma restrição cuja existência fica clara para o usuário. Para casos em que comparar editais é o objetivo, o seletor oferece uma quarta opção *"Todos os editais"*, que preserva o modo anterior.

5. Reordenação do conjunto de tools

A avaliação na Petrobras revelou um terceiro tipo de falha. À pergunta *"qual o período de inscrição?"*, o agente respondeu citando apenas a data da prova (20/02/2022) e afirmou que o edital não menciona período de inscrição. A inspeção mostrou que:

- A informação **está** no Anexo IV (cronograma): "17/12/2021 a 5/1/2022".
- A informação **está indexada** no ChromaDB como tabela do capítulo 20.
- A tool que o agente escolheu — `consultar_edital(editais_ativos, campo="cronograma")` — retornou apenas **1 evento**, porque a extração estruturada via LLM havia capturado um único item dos treze do cronograma.

5.1 O diagnóstico

Uma consulta direta ao SQLite confirmou:

```
conn.execute("SELECT dados_json FROM editais WHERE id = 3").fetchone()
# cronograma: 1 evento de 13
```

5.2 O problema arquitetural

Essa observação levou a uma reflexão sobre a arquitetura de ferramentas. As tools `consultar_edital` e `resumo_edital` operam sobre **artefatos pré-computados** durante a ingestão: o JSON estruturado validado por `EditaisInfo` (Pydantic) e o resumo focado em Ciência de Dados. Ambos dependem de o LLM da ingestão ter capturado a informação corretamente — uma fragilidade que o laudo 03 já havia documentado como *omissão seletiva*. Sempre que essa pré-extração falha, a tool devolve dado parcial ou ausente, e o agente, sem motivo para suspeitar, conclui que a informação não existe.

O RAG sobre Markdowns estruturais, em contraste, é uma representação **completa** do edital — depois das correções da seção 3, é demonstravelmente completo. Manter duas fontes paralelas sobre o mesmo edital, uma frágil e uma fiel, gera um modo de falha previsível: o agente escolhe a frágil e perde acesso à fiel.



Figura 7. As duas fontes em paralelo. A pré-extração é frágil porque depende de o LLM da ingestão ter capturado todos os campos; o RAG sobre Markdown é fiel ao documento. Manter as duas convida o agente a escolher a errada.

5.3 Decisão: remover as tools pré-computadas

A decisão foi remover `consultar_edital` e `resumo_edital` do conjunto de tools do agente. Uma única fonte de verdade — o RAG — passou a atender todas as perguntas factuais. O agente passou a operar com **cinco ferramentas**:

Ferramenta	Papel
<code>listar_editais</code>	Inventário; opcional quando o edital ativo está fixado
<code>buscar_no_edital</code>	Recuperação semântica via RAG; tool principal para qualquer fato específico
<code>listar_capitulos</code>	TOC do edital ativo
<code>ler_capitulo</code>	Leitura determinística de capítulo inteiro (cronograma, programa, etc.)
<code>data_hoje</code>	Data atual para comparações de vigência

O SYSTEM_PROMPT foi reescrito para refletir essa simplificação. `buscar_no_edital` passou a ser apresentada como tool principal e primeira escolha para qualquer fato. `ler_capitulo` é prescrita para perguntas sobre seções inteiras. A regra de fallback foi mantida e estreitada: se a busca não retornar resultado satisfatório, reformular a query e tentar novamente; se ainda assim falhar, usar `listar_capitulos` para identificar capítulos candidatos e ler o capítulo inteiro.

Após a remoção, a pergunta sobre o período de inscrição da Petrobras passou a ser respondida corretamente, com a comparação explícita de datas via `data_hoje`.

6. Self-check determinístico

6.1 Sintoma motivador

Mesmo após as correções acima, observou-se na avaliação que o agente ocasionalmente respondia perguntas factuais **sem chamar nenhuma ferramenta no turno corrente**, baseando-se exclusivamente no contexto da conversa anterior ou em conhecimento geral. Em alguns casos, a informação do turno anterior se aplicava à pergunta nova; em outros, não, e a resposta saía errada apesar de o RAG ter o dado.

6.2 Por que não forçar tool call no prompt

Forçar tool call por prompt é frágil — o usuário formulou explicitamente que não queria essa abordagem. A alternativa adotada foi inserir, fora do loop principal do agente, uma **camada determinística de verificação** acionada apenas quando o agente devolve resposta final sem nenhuma tool call no turno atual.

6.3 Como funciona

Essa camada faz uma segunda chamada ao LLM, sem ferramentas vinculadas, com um prompt focado em uma única decisão binária: a pergunta corrente exige consulta a alguma fonte do edital, ou pode ser respondida sem consulta? A resposta esperada é uma de duas tokens fixas: `PRECISA_CONSULTAR` ou `PODE_RESPONDER`.

As regras de decisão são explícitas no prompt:

1. O contexto anterior da conversa não é garantia para a pergunta atual.
2. Qualquer pergunta sobre fato específico do edital exige consulta.
3. Cumprimentos, agradecimentos e perguntas conversacionais não exigem.
4. Em caso de dúvida, exigir consulta.

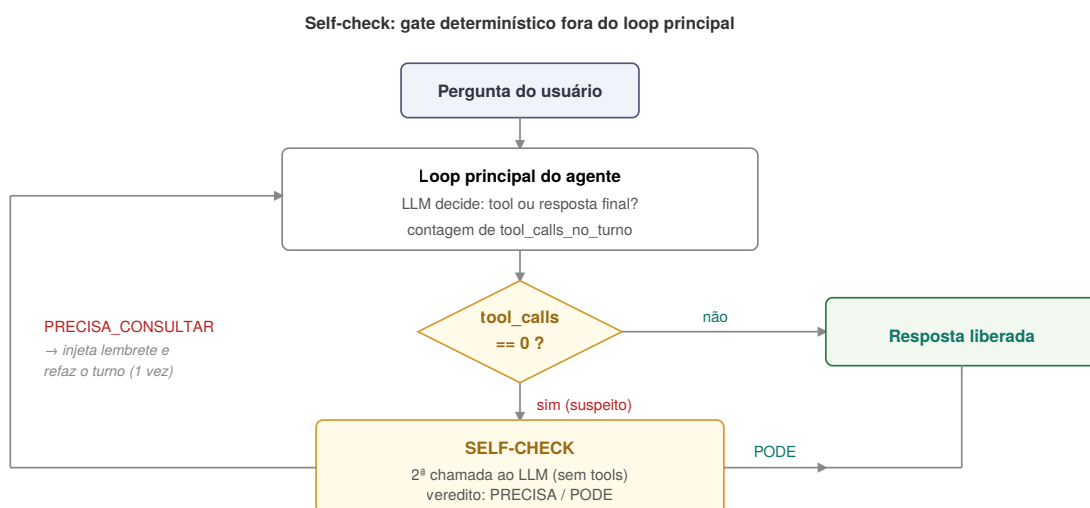


Figura 8. Fluxo do self-check. O gate só dispara quando o agente quer responder sem ter chamado nenhuma tool no turno. A reconsideração é forçada uma única vez — se o agente persistir, a resposta vai.

6.4 Restrição de uma volta

Se o veredito é `PRECISA_CONSULTAR`, o loop principal recebe uma `HumanMessage` interna pedindo reconsideração, e o agente é obrigado a uma nova rodada — agora com a expectativa de chamar tool. **O self-check dispara no máximo uma vez por turno**: se mesmo após o lembrete o agente insistir em responder sem tool, a resposta é aceita. Essa restrição evita laços infinitos e cobra do agente uma decisão final assumida.

A camada é determinística no sentido de que *a decisão de invocá-la* não depende do agente: é uma função do estado externo (zero tool calls). O conteúdo do veredito é probabilístico, naturalmente, porque é gerado por LLM. Mas o prompt é simples o suficiente para que a variância seja baixa: a tarefa é classificar a pergunta em uma de duas categorias bem definidas.

Em LangSmith, o self-check aparece como uma chamada adicional separada do loop principal, identificável pelo prompt único e pelo modo "sem tools". Isso facilita auditoria.

7. Limitações remanescentes

7.1 Alucinação contra contexto

Após as correções, o caso 3 da seção 2 — alternativas das provas objetivas da Petrobras — continua sendo respondido incorretamente. O retrieval traz o chunk do item 7.2 contendo o texto literal "*O julgamento de cada item será CERTO ou ERRADO*". O agente, ao gerar a resposta final, descarta o texto recuperado e responde "*5 alternativas, sendo apenas uma correta*". Há referência válida ao item 7.2 anexada à resposta — o que torna o erro auditável mas não impede sua emissão.

Esse é um caso clássico de **alucinação contra contexto** (Huang et al., 2025): o modelo possui um *prior* forte sobre o formato de provas de concurso e o aplica mesmo quando o contexto recuperado contradiz explicitamente esse prior. A defesa por prompt — instrução para releitura literal do trecho recuperado antes da resposta — foi tentada e não eliminou a falha; em alguns testes, a instrução piorou a qualidade geral das respostas, possivelmente por sobrecarregar o `SYSTEM_PROMPT`.

Este modo de falha é distinto dos três tratados nas seções 3 a 5. Não é problema de retrieval, não é problema de roteamento, não é problema de tool. É um comportamento do *gerador*. Tratá-lo provavelmente exige mecanismos fora do escopo deste laudo: avaliação automática de fidelidade resposta-vs-trechos via segundo LLM julgador, ou modelagem da resposta como extração estruturada vinculada ao trecho citado.

7.2 Granularidade da referência citada

O metadado `numero_secao` de cada chunk corresponde ao **primeiro item do bloco**. Quando vários itens curtos foram fundidos por `_fundir_pequenos`, os subseqüentes ficam visíveis no texto do chunk mas não nos metadados. A citação no fim da resposta é gerada pelo LLM lendo o texto, e tem a precisão que o LLM consegue extrair — não há garantia determinística.

7.3 Variância entre modelos

Toda a discussão acima é específica do modelo em uso (`gpt-4o-mini` , na configuração padrão). A robustez observada com modelos maiores não foi medida sistematicamente nesta rodada.

8. Estado atual e próximos passos

A branch `develop` ao fim deste ciclo consolida:

- **chunker estrutural** com princípio invariante de preservação total — nenhum texto do Markdown é descartado em nenhum caso;
- **fixação do edital ativo** via interface, com reset automático de histórico ao trocar;
- **conjunto de tools reduzido a cinco**, com `buscar_no_edital` e `ler_capitulo` como ferramentas factuais primárias;
- **self-check determinístico** para o caso de resposta sem tool call;
- **referências obrigatórias** ao final de cada resposta factual no formato `ref: edital [X.Y, Z.W]` , geradas pelo agente a partir do metadado `numero_secao` injetado nos resultados do RAG.

Direções identificadas para trabalho futuro:

1. **Bateria de avaliação multi-edital pós-correções.** A bateria do laudo 04 cobria apenas o BNDES. Uma bateria nova, com perguntas equivalentes adaptadas a cada um dos três editais e respostas de referência manualmente extraídas, é o próximo passo natural para mensurar o efeito agregado das mudanças.
2. **Tratamento da alucinação contra contexto** (limitação da seção 7.1). Um caminho promissor é uma camada de validação resposta-vs-trechos, aplicada após a geração final, que verifique se as afirmações factuais da resposta estão contidas nos chunks recuperados — análoga ao self-check da seção 6, mas focada em fidelidade ao contexto e não em decisão de consulta.
3. **Calibração dos parâmetros do chunker.** Os valores atuais (`CHUNK_MIN=400` , `CHUNK_MAX=3500` , `CHUNK_ALVO=1500`) foram herdados do laudo 07 e não foram revistos após a remoção do filtro de descarte. É plausível que, com a base de chunks agora completa, valores diferentes resultem em retrieval melhor.

9. Referências

Huang, L. et al. (2025). *A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions*. ACM Transactions on Information Systems, 43(5), 1-55. DOI: 10.1145/3703155.

Liu, N. F. et al. (2024). *Lost in the Middle: How Language Models Use Long Contexts*. Transactions of the Association for Computational Linguistics, 12, 157-173. DOI: 10.1162/tacl_a_00638.